Distributed Anti-Money Laundering Transaction Analysis and Classification Using Apache Spark

INT315 – Cluster Computing Project Report

Cover information	Value
Student name	[Enter student name]
Registration number	[Enter registration number]
Section	[Enter section]
University	[Enter university]
Faculty name	[Enter faculty name]
Submission date	[Enter submission date]

Educational analysis of simulated AMLSim transactions; not a deployed banking, real-time monitoring, or automated decision system.

Table of Contents

- 1. Project Title
- 2. Problem Statement
- 3. Objectives and Scope
- 4. Dataset Description
- 5. Dataset Source
- 6. Tools and Technologies Used
- 7. Implementation / Methodology
- 8. Notebook Code
- 9. Notebook Execution Outputs
- 10. Output / Result Screenshots
- 11. Results and Discussion
- 12. Conclusion and Future Work
- References

1. Project Title

Distributed Anti-Money Laundering Transaction Analysis and Classification Using Apache Spark. Course: INT315 – Cluster Computing.

2. Problem Statement

Large transaction data requires repeatable preparation, aggregation, and evaluation. Rare positive labels make accuracy alone insufficient. This project demonstrates an offline Spark workflow on synthetic AMLSim data and compares precision, recall, F1, PR-AUC, ROC-AUC, and confusion counts.

AMLSim labels represent simulator-generated activity, not verified real-world crime. Notebook scores are not proof of laundering or recommendations for real-world action.

3. Objectives and Scope

- Prepare and validate a reproducible transaction sample.
- Demonstrate Spark DataFrame transformations and Spark SQL summaries.
- Compare Decision Tree, Random Forest, and Gradient-Boosted Tree classifiers.
- Use a chronological train/validation/test split and report test results.
- Display tables, metrics, previews, and plots inline in the notebook.

The notebook imports existing project modules for pipeline operations. Only notebook code cells are reproduced here; standalone modules are not appended.

4. Dataset Description

The notebook reports 500,000 prepared transaction rows and eight prepared columns. The saved class distribution is:

class_name	count	percentage
AML-negative	499490	99.8980
AML-positive	510	0.1020

Class imbalance is visible in the saved table. The input header has 11 columns, including 2 source columns named Account that are resolved by position.

5. Dataset Source

The notebook identifies IBM AMLSim HI-Small transaction data. Its saved output states that the source CSV and the existing 500,000-row sample were available; the sample was reused and source-row counts were not recalculated in this run.

IBM AMLSim: github.com/IBM/AMLSim; example data: AMLSim Wiki.

6. Tools and Technologies Used

Technology	Role shown in the notebook
Python 3.10.0	Notebook orchestration and analysis code.
Apache Spark / PySpark 3.5.9	Local Spark DataFrames, SQL, and Spark ML.
Spark local[2]	Two local worker threads on one machine.
Pandas / NumPy	Compact metrics, tables, and analysis.
Matplotlib / Seaborn	Inline notebook visualizations.
Jupyter	Code cells, narrative, tabular outputs, and plots.

The notebook records Python 3.10.0, Spark 3.5.9, and master local[2].

7. Implementation / Methodology

7.1 Notebook setup

The notebook locates the project, imports its modules, and provides controls for sampling, training, and SQL queries.

7.2 Loading and preparation

It displays the source header, prepared schema, row count, sample preview, and null summary.

7.3 Sample profile

Class distribution and amount-range summaries are calculated and plotted inline.

7.4 SQL analysis

Seven queries cover transaction values, active senders, payment types, aggregate statistics, a sender join, wildcard search, and AML rates by payment type.

7.5 Modeling and evaluation

Three classifiers are evaluated chronologically. Threshold-based metrics and continuous-score ranking metrics are both reported.

7.6 Output handling

Notebook tables and figures are inline; the final cell releases cached data and stops Spark.

The report generator reads embedded notebook outputs only. It does not execute Spark or load model metrics, SQL CSVs, or charts from the artifacts directory.

8. Notebook Code

Source below is taken only from non-empty code cells in notebooks\01_aml_spark_workflow.ipynb. Pipeline logic imported from other project files is not copied into this report.

Notebook code cell 3

```
import csv
import os
import sys
import tempfile
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from IPython.display import display
from pyspark import StorageLevel
from pyspark.sql import SparkSession, functions as F

configured_root = os.environ.get("AML_SPARK_PROJECT_DIR")
root_candidates = (
    ([Path(configured_root)] if configured_root else [])
    + [Path.cwd(), *Path.cwd().parents]
)
PROJECT_DIR = next(
    (path.resolve() for path in root_candidates if (path / "main.py").is_file()),
    None,
)
if PROJECT_DIR is None:
    raise FileNotFoundError(
        "Could not find main.py in the current directory or its parents. "
        "Launch Jupyter from the project tree or set AML_SPARK_PROJECT_DIR."
    )

project_path = str(PROJECT_DIR)
if project_path not in sys.path:
    sys.path.insert(0, project_path)

from data_processing import add_features, create_stratified_sample, load_and_clean
from ml_models import train_and_evaluate_models
from spark_sql import run_sql_queries
from visualizations import _curve_points

DATA_PATH = PROJECT_DIR / "data" / "HI-Small_Trans.csv"
SAMPLE_PATH = PROJECT_DIR / "data" / "HI-Small_Trans_sample_500000.csv"
OUTPUT_DIR = PROJECT_DIR / "artifacts" / "hi-small-500k"
SAMPLE_SIZE = 500_000
SAMPLE_SEED = 42

# Training and SQL run so their fresh outputs appear in notebook cells.
# Sampling stays off to reuse the already-generated sample.
RUN_SAMPLING = False
RUN_MODEL_TRAINING = True
RUN_SQL_QUERIES = True

print(f"Project root: {PROJECT_DIR}")
print(f"Python executable: {sys.executable}")
print(f"Python version: {sys.version.split()[0]}")
print(f"Source CSV available: {DATA_PATH.is_file()}")
```

```
print(f"Sample available: {SAMPLE_PATH.is_file()}")
print("Run controls:", {
    "RUN_SAMPLING": RUN_SAMPLING,
    "RUN_MODEL_TRAINING": RUN_MODEL_TRAINING,
    "RUN_SQL_QUERIES": RUN_SQL_QUERIES,
})
```

Notebook code cell 5

```
spark = (
    SparkSession.builder.appName("Simple IBM AML Analysis")
    .master("local[2]")
    .config("spark.sql.shuffle.partitions", "8")
    .config("spark.default.parallelism", "2")
    .getOrCreate()
)
spark.sparkContext.setLogLevel("WARN")
print(f"Spark version: {spark.version}")
print(f"Spark master: {spark.sparkContext.master}")
```

Notebook code cell 7

```
header_path = DATA_PATH if DATA_PATH.is_file() else SAMPLE_PATH
if not header_path.is_file():
    raise FileNotFoundError(
        "Neither the source CSV nor the prepared sample was found. Place "
        "HI-Small_Trans.csv in data\\, or provide the existing sample."
    )

with header_path.open(encoding="utf-8-sig", newline="") as csv_file:
    source_header = next(csv.reader(csv_file), None)
if not source_header:
    raise ValueError(f"No CSV header found in {header_path}")

print(f"Header read from: {header_path.name}")
print(f"Column count: {len(source_header)}")
display(pd.DataFrame({"column_position": range(len(source_header)), "column_name": source_header}))
```

Notebook code cell 8

```
source_stats = {}
if RUN_SAMPLING:
    if not DATA_PATH.is_file():
        raise FileNotFoundError(f"Cannot sample because the original CSV is missing: {DATA_PATH}")
    selected_sample = Path(
        create_stratified_sample(
            spark,
            str(DATA_PATH),
            str(SAMPLE_PATH),
            SAMPLE_SIZE,
            SAMPLE_SEED,
            source_stats,
        )
    )
    print(f"Created reproducible sample: {selected_sample}")
else:
    selected_sample = SAMPLE_PATH
    if not selected_sample.is_file():
        raise FileNotFoundError(
            f"Prepared sample not found: {selected_sample}. Add the sample file "
            "or set RUN_SAMPLING = True and provide the original source CSV."
        )
    print(f"Reusing existing sample: {selected_sample}")
    print("Sampling was not rerun; source class counts are not recalculated in this mode.")
```

Notebook code cell 10

```
if "transactions" in globals() and transactions is not None:
    transactions.unpersist()
transactions = add_features(load_and_clean(spark, str(selected_sample))).persist(
    StorageLevel.DISK_ONLY
)
transaction_count = transactions.count()
if transaction_count != SAMPLE_SIZE:
    raise ValueError(
        f"Expected {SAMPLE_SIZE:,} valid prepared rows, got {transaction_count:,}."
    )
print(f"Prepared rows: {transaction_count:,}")
print(f"Prepared columns: {len(transactions.columns)}")
transactions.printSchema()
display(transactions.limit(8).toPandas())
```

Notebook code cell 11

```
null_summary = transactions.agg(
    F.count("*").alias("prepared_rows"),
    *[
        F.sum(F.when(F.col(column).isNull(), 1).otherwise(0)).alias(f"{column}_nulls")
        for column in transactions.columns
    ],
).first().asDict()
display(pd.DataFrame([null_summary]).T.rename(columns={0: "count"}))
```

Notebook code cell 13

```
class_distribution = (
    transactions.groupBy("is_laundering")
    .count()
    .orderBy("is_laundering")
```

```
    .toPandas()
)
class_distribution["class_name"] = class_distribution["is_laundering"].map(
    {0: "AML-negative", 1: "AML-positive"}
)
class_distribution["percentage"] = (
    class_distribution["count"] / class_distribution["count"].sum() * 100
)
display(class_distribution[["class_name", "count", "percentage"]])

fig, ax = plt.subplots(figsize=(7, 4))
sns.barplot(data=class_distribution, x="class_name", y="count", hue="class_name", legend=False, ax=ax)
for bar, row in zip(ax.patches, class_distribution.itertuples()):
    ax.annotate(
        f"{row.count:,} ({row.percentage:.2f}%)",
        (bar.get_x() + bar.get_width() / 2, bar.get_height()),
        ha="center",
        va="bottom",
        xytext=(0, 4),
        textcoords="offset points",
    )
ax.set(title="Prepared transaction labels", xlabel="", ylabel="Transactions")
fig.tight_layout()
plt.show()
plt.close(fig)
```

Notebook code cell 15

```
feature_summary = (
    transactions.groupBy("is_near_threshold")
    .agg(
        F.count("*").alias("transaction_count"),
        F.min("amount").alias("minimum_amount"),
        F.avg("amount").alias("average_amount"),
        F.max("amount").alias("maximum_amount"),
    )
    .orderBy("is_near_threshold")
    .toPandas()
)
display(feature_summary)
```

Notebook code cell 17

```
model_metrics = None
if RUN_MODEL_TRAINING:
    # The existing helper writes its metrics CSV; keep that side effect temporary
    # so this notebook displays results without adding files to project artifacts.
    with tempfile.TemporaryDirectory(prefix="aml-spark-notebook-") as temporary_output:
        model_metrics = train_and_evaluate_models(
            transactions,
            temporary_output,
            source_stats=source_stats or None,
            sample_size=SAMPLE_SIZE,
            sample_seed=SAMPLE_SEED,
        )
    print("Training complete. Temporary metrics files were removed.")

    preview_rows = 20
    for result in model_metrics:
        probabilities = np.asarray(result["_test_probabilities"], dtype=float)
        actual_labels = np.asarray(result["_test_labels"], dtype=int)
        predicted_labels = (probabilities >= float(result["threshold"])).astype(int)
        prediction_preview = pd.DataFrame({
            "test_row_order": np.arange(1, min(preview_rows, len(actual_labels)) + 1),
            "actual_label": actual_labels[:preview_rows],
            "predicted_label": predicted_labels[:preview_rows],
            "positive_probability": probabilities[:preview_rows],
        })
        print(f"{result['model']} ? first {len(prediction_preview)} test predictions")
        display(prediction_preview)
else:
    print("Model training skipped. Set RUN_MODEL_TRAINING = True to train all three models.")
```

Notebook code cell 19

```
if RUN_SQL_QUERIES:
    sql_results = run_sql_queries(spark, transactions)
    for query_name, result in sql_results.items():
        # The existing queries return at most 20 rows or small aggregate tables.
        result_table = result.toPandas()
        print(f"{query_name}: {len(result_table):,} rows")
        display(result_table)
else:
    print("SQL execution skipped. Set RUN_SQL_QUERIES = True to run the seven queries.")
```

Notebook code cell 21

```
if model_metrics is None:
    raise RuntimeError(
        "Fresh metrics are not available. Run the model-training cell first; "
        "this notebook intentionally does not load old saved scores."
    )

metric_columns = [
    "model", "accuracy", "precision", "recall", "f1", "pr_auc", "roc_auc",
    "tp", "tn", "fp", "fn", "threshold",
]
metrics_table = pd.DataFrame(
    [{column: result[column] for column in metric_columns} for result in model_metrics]
```

```
)
split_stats = model_metrics[0]["_split_stats"]
print("Metrics are from the models trained in this notebook run.")
display(
    metrics_table.style.format({
        "accuracy": "{:.4f}", "precision": "{:.4f}",
        "recall": "{:.4f}", "f1": "{:.4f}",
        "pr_auc": "{:.4f}", "roc_auc": "{:.4f}",
        "threshold": "{:.4f}",
    }).set_caption("Fresh chronological test-set model comparison")
)

split_table = pd.DataFrame([
    {
        "split": split_name.title(),
        **counts,
        "positive_rate": counts["positive"] / counts["rows"],
    }
    for split_name, counts in split_stats.items()
])
display(split_table.style.format({"positive_rate": "{:.4%}"}).set_caption("Actual chronological split sizes"))
```

Notebook code cell 22

```
if metrics_table is not None:
    fig, axes = plt.subplots(1, len(metrics_table), figsize=(5 * len(metrics_table), 4))
    if len(metrics_table) == 1:
        axes = [axes]
    for axis, row in zip(axes, metrics_table.itertuples(index=False)):
        confusion = pd.DataFrame(
            [[row.tn, row.fp], [row.fn, row.tp]],
            index=["Actual negative", "Actual positive"],
            columns=["Predicted negative", "Predicted positive"],
        )
        sns.heatmap(confusion, annot=True, fmt=",d", cmap="Blues", cbar=False, ax=axis)
        axis.set_title(row.model)
        axis.set_xlabel("Prediction")
        axis.set_ylabel("Actual")
    fig.suptitle("Test confusion matrices at each model's validation-selected threshold")
    fig.tight_layout()
    plt.show()
    plt.close(fig)

    comparison = metrics_table.melt(
        id_vars="model",
        value_vars=["accuracy", "precision", "recall", "f1", "pr_auc", "roc_auc"],
        var_name="metric",
        value_name="score",
    )
    fig, ax = plt.subplots(figsize=(11, 5))
    sns.barplot(data=comparison, x="metric", y="score", hue="model", ax=ax)
    ax.set(title="Model comparison on chronological test data", xlabel="Metric", ylabel="Score", ylim=(0, 1))
    ax.legend(title="Model", bbox_to_anchor=(1.02, 1), loc="upper left")
    fig.tight_layout()
    plt.show()
    plt.close(fig)

    best_f1_model = metrics_table.loc[metrics_table["f1"].idxmax(), "model"]
    print(f"Highest test F1 in the displayed metrics: {best_f1_model}")
    print("Interpret precision/recall together with prevalence and review capacity; accuracy alone is not
evidence of useful AML detection.")
```

Notebook code cell 24

```
if model_metrics is None:
    raise RuntimeError("Run model training before generating current-run plots.")

sns.set_theme(style="whitegrid")

# 1. Correlation heatmap, matching the project's bounded 20,000-row sample.
correlation_sample = (
    transactions.select("amount", "is_near_threshold")
    .sample(False, 0.1, seed=42)
    .limit(20_000)
    .toPandas()
)
if correlation_sample.empty:
    correlation_sample = transactions.select("amount", "is_near_threshold").limit(20_000).toPandas()
fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(correlation_sample.corr(numeric_only=True), annot=True, cmap="coolwarm", center=0, ax=ax)
ax.set_title("Correlation of numeric transaction features")
fig.tight_layout()
plt.show()
plt.close(fig)

# 2?3. Curves use this run's own chronological test labels and probabilities.
for curve_name in ("precision_recall", "roc"):
    fig, ax = plt.subplots(figsize=(7, 5))
    for result in model_metrics:
        recall, precision, fpr, tpr = _curve_points(
            result["_test_labels"], result["_test_probabilities"]
        )
        if curve_name == "precision_recall":
            ax.plot(recall, precision, label=result["model"])
        else:
            ax.plot(fpr, tpr, label=result["model"])
    if curve_name == "precision_recall":
        prevalence = float(np.mean(model_metrics[0]["_test_labels"]))
        ax.axhline(prevalence, linestyle="--", color="gray", label=f"Test prevalence ({prevalence:.3%})")
        ax.set(title="Precision-recall curves ? chronological test", xlabel="Recall", ylabel="Precision")
```

```
    else:
        ax.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Chance")
        ax.set(title="ROC curves ? chronological test", xlabel="False positive rate", ylabel="True positive
rate")
    ax.set(xlim=(0, 1), ylim=(0, 1))
    ax.legend(loc="best")
    fig.tight_layout()
    plt.show()
    plt.close(fig)

# 4. Random Forest feature importance from the fitted model result.
forest_result = next(result for result in model_metrics if "_feature_importances" in result)
importance = pd.DataFrame([
    {"feature": feature, "importance": value}
    for feature, value in forest_result["_feature_importances"].items()
]).sort_values("importance", ascending=False)
fig, ax = plt.subplots(figsize=(8, 4))
sns.barplot(data=importance, x="importance", y="feature", hue="feature", legend=False, ax=ax)
ax.set(title="Random Forest feature importance", xlabel="Importance", ylabel="Feature")
fig.tight_layout()
plt.show()
plt.close(fig)

# 5?6. Side-by-side metric comparisons from the current test results.
for columns, title in (
    (["accuracy", "pr_auc", "roc_auc"], "Accuracy and ranking metrics"),
    (["precision", "recall", "f1"], "Precision, recall, and F1"),
):
    chart_data = metrics_table.melt(id_vars="model", value_vars=columns, var_name="metric",
value_name="score")
    fig, ax = plt.subplots(figsize=(9, 5))
    sns.barplot(data=chart_data, x="metric", y="score", hue="model", ax=ax)
    ax.set(title=title, xlabel="Metric", ylabel="Score", ylim=(0, 1))
    ax.legend(title="Model")
    fig.tight_layout()
    plt.show()
    plt.close(fig)

# 7. Confusion matrices at the validation-selected thresholds.
fig, axes = plt.subplots(1, len(model_metrics), figsize=(5 * len(model_metrics), 4))
if len(model_metrics) == 1:
    axes = [axes]
for ax, result in zip(axes, model_metrics):
    matrix = pd.DataFrame(
        [[result["tn"], result["fp"]], [result["fn"], result["tp"]]],
        index=["Actual negative", "Actual positive"],
        columns=["Predicted negative", "Predicted positive"],
    )
    sns.heatmap(matrix, annot=True, fmt=",d", cmap="Blues", cbar=False, ax=ax)
    ax.set_title(result["model"])
    ax.set_xlabel("Prediction")
    ax.set_ylabel("Actual")
fig.suptitle("Confusion matrices ? chronological test")
fig.tight_layout()
plt.show()
plt.close(fig)
print("All charts were rendered inline and were not saved to files.")
```

Notebook code cell 27

```
if "transactions" in globals() and transactions is not None:
    transactions.unpersist()
if "spark" in globals() and spark is not None:
    spark.stop()
    print("Spark session stopped and prepared transaction cache released.")
```

9. Notebook Execution Outputs

These are text outputs embedded in the notebook, not recreated terminal screenshots. This report does not rerun the workflow.

Output from notebook cell 3

```
Project root: <project root>
Python version: 3.10.0
Source CSV available: True
Sample available: True
Run controls: {'RUN_SAMPLING': False, 'RUN_MODEL_TRAINING': True, 'RUN_SQL_QUERIES': True}
```

Output from notebook cell 5

```
Spark version: 3.5.9
Spark master: local[2]
```

Output from notebook cell 7

```
Header read from: HI-Small_Trans.csv
Column count: 11
column_position         column_name
0                  0           Timestamp
1                  1           From Bank
2                  2             Account
3                  3             To Bank
4                  4             Account
5                  5     Amount Received
6                  6  Receiving Currency
7                  7         Amount Paid
```

```
8                8    Payment Currency
9                9      Payment Format
10              10       Is Laundering
```

Output from notebook cell 8

```
Reusing existing sample: <project root>\data\HI-Small_Trans_sample_500000.csv
Sampling was not rerun; source class counts are not recalculated in this mode.
```

Output from notebook cell 10

```
Prepared rows: 500,000
Prepared columns: 8
root
 |-- transaction_id: string (nullable = true)
 |-- timestamp: timestamp (nullable = true)
 |-- sender_account: string (nullable = true)
 |-- receiver_account: string (nullable = true)
 |-- amount: double (nullable = true)
 |-- payment_type: string (nullable = true)
 |-- is_laundering: integer (nullable = true)
 |-- is_near_threshold: integer (nullable = true)
transaction_id           timestamp  \
0  105dcf63600629bd34f815df0349202a8231cc92d6d8d1... 2022-09-10 01:51:00
1  c9190d0bca26130f5fe913ee44d84bdf59a75e88e9af5f... 2022-09-02 20:08:00
2  a092e547d7f33382454227b623543cef263b23d58982ae... 2022-09-05 22:57:00
3  8d93756e2048f6153d7ccef5bcc9f0dfda5ba94608d24f... 2022-09-01 19:08:00
4  4147b8620ba26a48299b03ef20f68c12a38dbcc57bfcd1... 2022-09-01 23:51:00
5  a3639725d52a3fc5708ac7f1823bb8de838e2e5b533e29... 2022-09-08 18:10:00
6  4a51a87a7c808a60c3cab92031ef61a1dcb48b63cd548f... 2022-09-06 00:23:00
7  133ad7709850edf24860bd7a8944ba904a1c8a96f206b6... 2022-09-01 07:30:00

      sender_account   receiver_account     amount  payment_type  \
0  0045304:8115860C0  0046906:811588F30    8885.03           ACH
1    01467:800268E80   001068:800F74A20     183.56   Credit Card
2   003229:8066F3960   001412:806C10A70    3120.89   Credit Card
3    00701:8004E4020    00701:8004E4020   30046.41  Reinvestment
4   029467:80390ABF0  0212156:80C40AFE0  113557.21   Credit Card
5      015:8039DA9B0      015:803F26EA0  684213.18           ACH
6   016224:808D6E8C0   005140:80D1FCB30      11.08   Credit Card
7      070:100428978     0118:80DC09890      30.73        Cheque

   is_laundering  is_near_threshold
0              0                  0
1              0                  0
2              0
```

Output from notebook cell 11

```
count
prepared_rows            500000
transaction_id_nulls          0
timestamp_nulls               0
sender_account_nulls          0
receiver_account_nulls        0
amount_nulls                  0
payment_type_nulls            0
is_laundering_nulls           0
is_near_threshold_nulls       0
```

Output from notebook cell 17

```
DATA USAGE REPORT (before model training)
Source row count and source class counts: unavailable.
Sample row count: 500,000; requested size: 500,000; seed: 42.
Sample AML-positive: 510 (0.1020%); AML-negative: 499,490 (99.8980%).
Train: 299,730 rows; AML-positive=212 (0.0707%); AML-negative=299,518.
Validation: 99,881 rows; AML-positive=103 (0.1031%); AML-negative=99,778.
Test: 100,389 rows; AML-positive=195 (0.1942%); AML-negative=100,194.
Model features (3): amount, is_near_threshold, payment_type_index
DecisionTreeClassifier: 100,389 test rows to evaluate; training time pending.
RandomForestClassifier: 100,389 test rows to evaluate; training time pending.
GBTClassifier: 100,389 test rows to evaluate; training time pending.
Total runtime: in progress.
Split policy: chronological by timestamp (60% train, 20% validation, 20% test quantiles). Positive-rate
differences are measured temporal distribution shift, not stratified away; timestamps and labels are excluded
from model features.
Leakage controls: account IDs and transaction IDs are excluded; the payment-type index is fit on training
rows only; class weights use training labels only.
Training DecisionTreeClassifier...
  Validation-selected threshold=0.902052; test precision=0.0170, recall=0.9385, F1=0.0335, PR-AUC=0.0165, ROC-
AUC=0.9129, accuracy=0.8947.
  Test confusion counts: TP=183, TN=89634, FP=10560, FN=12.
Training RandomForestClassifier...
  Validation-selected threshold=0.907338; test precision=0.0472, recall=0.4718, F1=0.0857, PR-AUC=0.0315, ROC-
AUC=0.9428, accuracy=0.9805.
  Test confusion counts: TP=92, TN=98335, FP=1859, FN=103.
Training GBTClassifier...
  Validation-selected threshold=0.875354; test precision=0.0332, recall=0.6256, F1=0.0631, PR-AUC=0.0291, ROC-
AUC=0.9368, accuracy=0.9639.
  Test confus
```

Output from notebook cell 19

```
high_value_transactions: 20 rows
transaction_id     sender_account  \
0   e0565449185c62fec25a8a55dbc168a59ced725410b45a...      004:806B9AF90
1   2762ab5f796daf6a9637530dd4b90dc775e44e300ce097...   015964:8026E7B20
2   06ebe610f53707475e67659429d56a7d9e323ae80b1239...   014433:8023B9740
```

```
3   117dea01ede0fcea4b9c46cfb4530e977d0a1b15847513...  0112497:804BB7A20
4   e47956ae7a3919c3c5ad8500b3da1af98baca506fca672...   029467:8039E0E70
5   9d19715c1a3a99b12b0a3d251b040002bef086371b76c7...  0133686:80C776BF0
6   b07882e01f551791debb23beb7576346eff2b194bca509...  0010060:804033940
7   bd65b78117d882530f8ba954f08490d63720b40876edda...  0010060:804033940
8   8562c07ce120fd7b17c2a039d2bfc1512dece669fa9c05...  0010057:803B18040
9   d9c9c1348a42229b0698b4ef1cedee88adfa7f9e4fc12d...  0017554:806BFCF10
10  953e42e08ce254cef3c023ba898695805968ffa0169982...  0016787:806660BC0
11  cea9a862f1a1a590fbfc88692b83baa630e60550db248d...  0212789:804B8F230
12  af36815beec6381f173b5aefe96cfce45fbecc37b88ee2...  0013157:804CB0D10
13  8bf219782f45af7b3ea3f1ac2057e661dffddb5183f001...  0016927:806537530
14  802fca9a6bbaa842050bcefb579dae7170af69caa090e1...  0112637:8053C22F0
15  fa0cd405d5fc0f00ac8f3ba962a3a26e9a60dabfc5be17...      015:80368FF60
16  cb1d6aa5390e19a37b4a90a3fd3eb28232c3333e3b0e40...      016:805424040
17  2ff80b229ab99e7f85373d3be555ece7eb28bb0e40715e...     0287:8060BF530
18  1865c4c5ebc426db5576fc03ffebc1ed51f8b7820c88c6...  0010813:80462D300
19  f5c981032ad7f9f77929984ab85143a3b0fc03cfc5ec06...    01292:800DAF690

     receiver_account        amount payment_type
0   0116781:806B9B6A0  8.485314e+10          ACH
1   0114590:805FBE280  6.943162e+10       Cheque
2       021:80368E630  5.867878e+10         Cash
3   0
```

Output from notebook cell 21

```
Metrics are from the models trained in this notebook run.
<pandas.io.formats.style.Styler at 0x2921365f970>
<pandas.io.formats.style.Styler at 0x292137d18a0>
```

Output from notebook cell 22

```
<Figure size 1500x400 with 3 Axes>
<Figure size 1100x500 with 1 Axes>
Highest test F1 in the displayed metrics: RandomForestClassifier
Interpret precision/recall together with prevalence and review capacity; accuracy alone is not evidence of
useful AML detection.
```

Output from notebook cell 24

```
<Figure size 500x400 with 2 Axes>
<Figure size 700x500 with 1 Axes>
<Figure size 700x500 with 1 Axes>
<Figure size 800x400 with 1 Axes>
<Figure size 900x500 with 1 Axes>
<Figure size 900x500 with 1 Axes>
<Figure size 1500x400 with 3 Axes>
All charts were rendered inline and were not saved to files.
```

Output from notebook cell 27

```
Spark session stopped and prepared transaction cache released.
```

10. Output / Result Screenshots

The figures below are actual inline PNG outputs embedded in the notebook. They are not desktop screenshots and are not loaded from the artifacts folder.

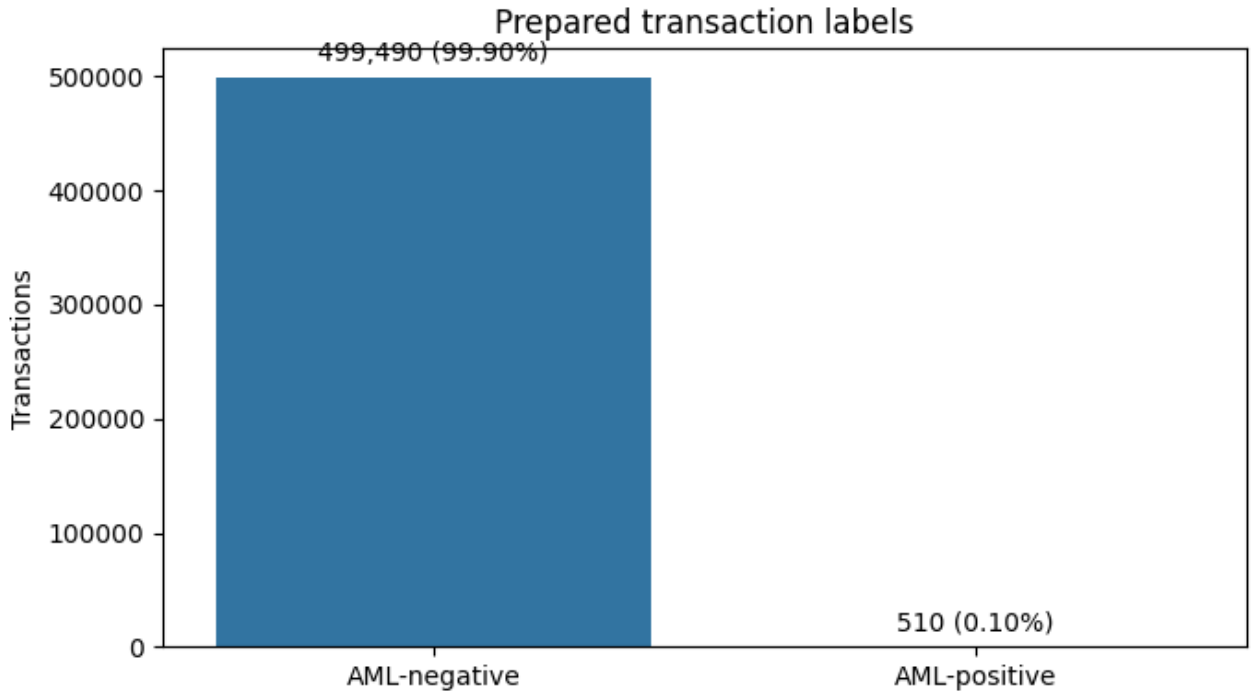


Figure 1. Inline chart output from notebook cell 13.

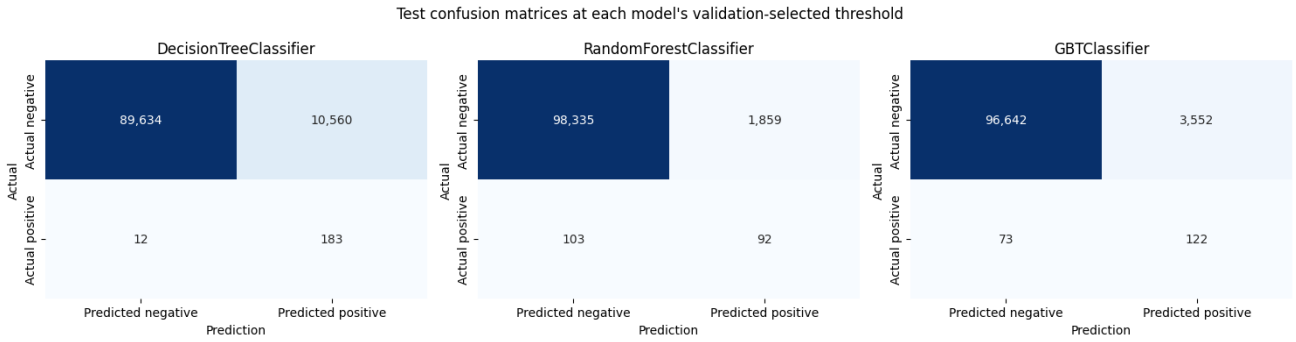


Figure 2. Inline chart output from notebook cell 22.

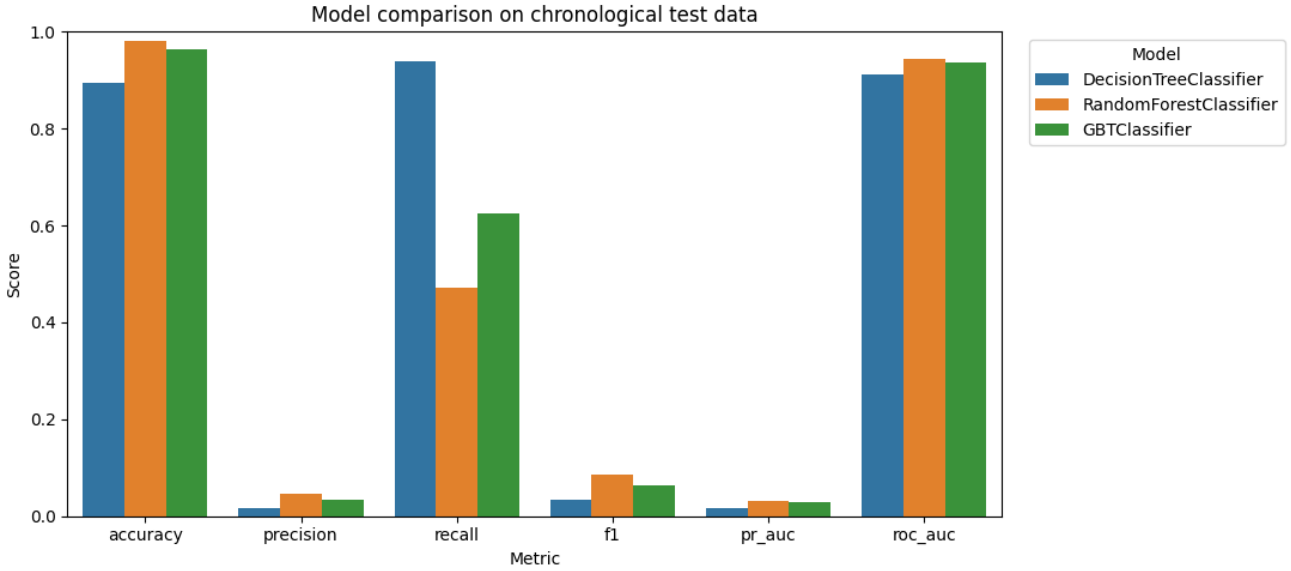


Figure 3. Inline chart output from notebook cell 22.

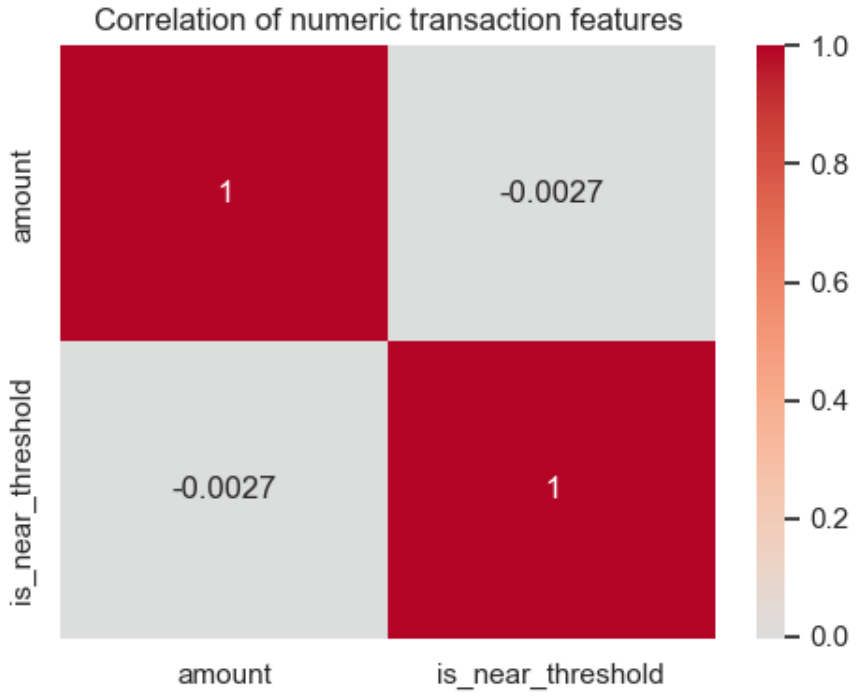


Figure 4. Inline chart output from notebook cell 24.

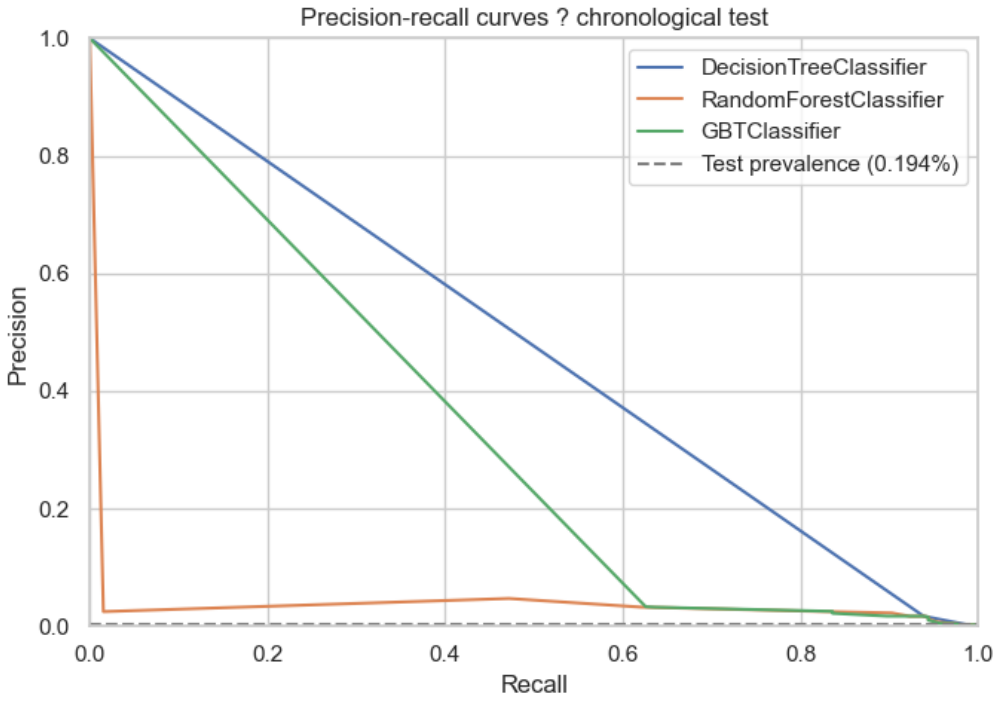


Figure 5. Inline chart output from notebook cell 24.

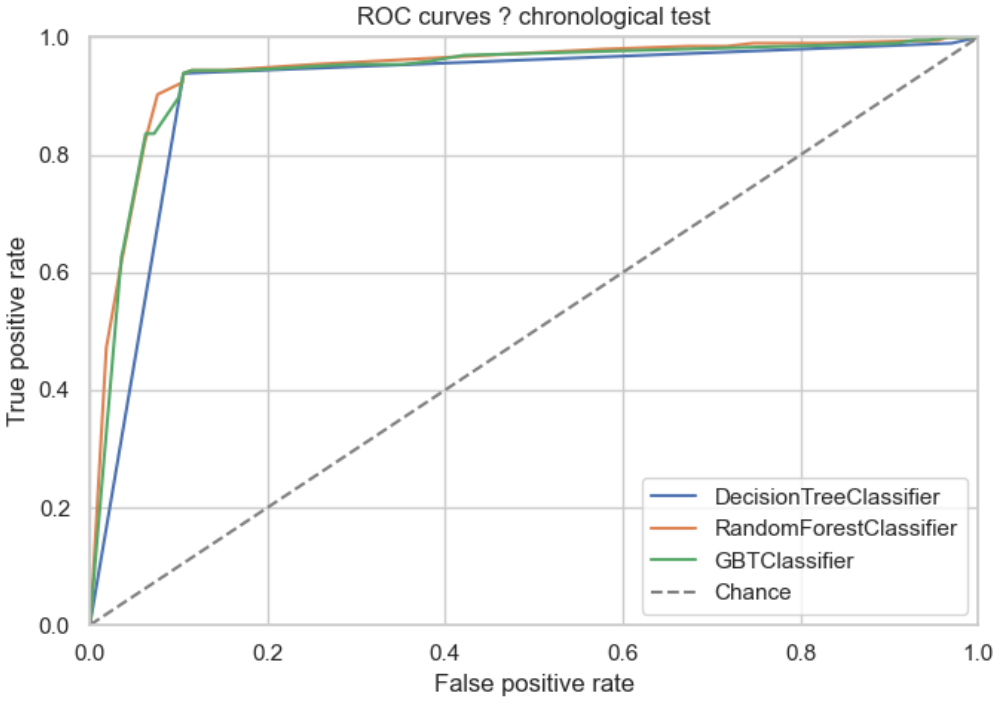


Figure 6. Inline chart output from notebook cell 24.

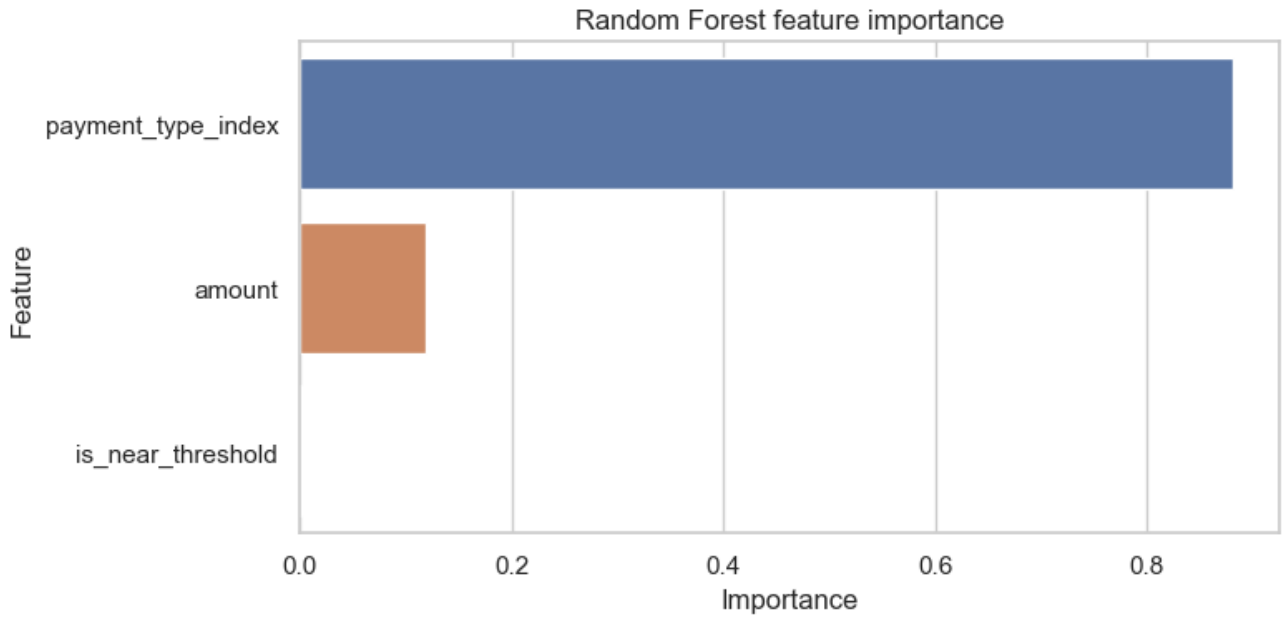


Figure 7. Inline chart output from notebook cell 24.

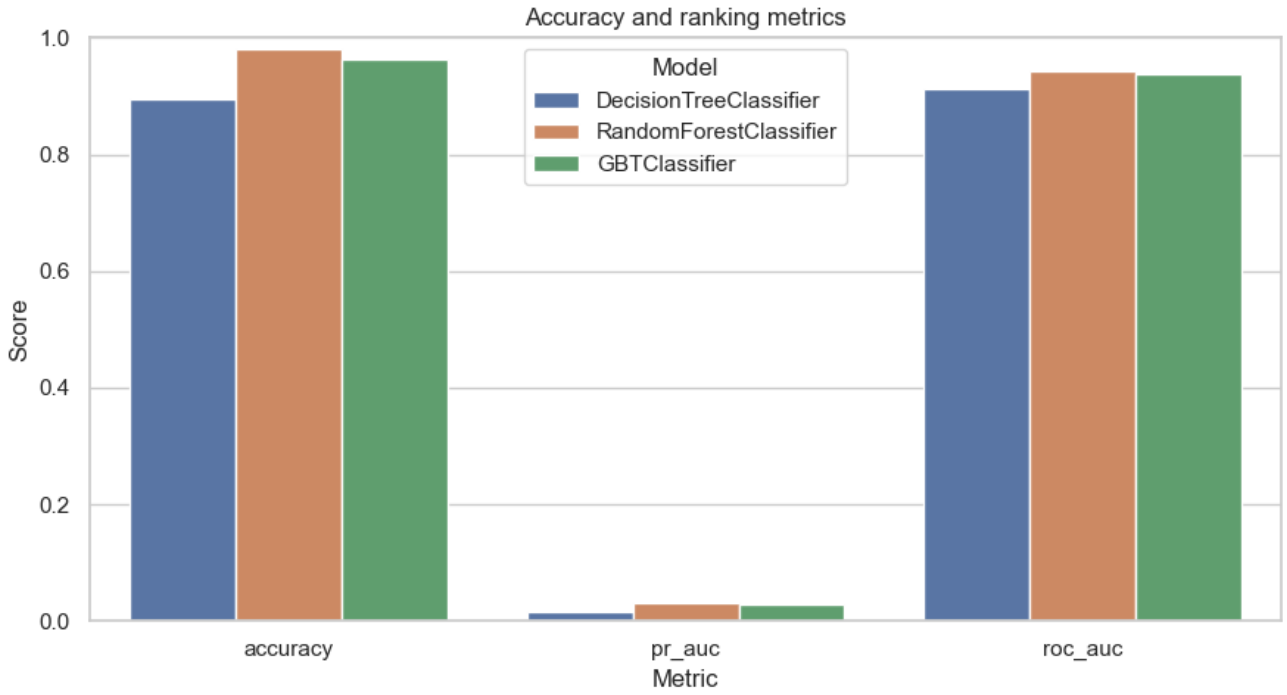


Figure 8. Inline chart output from notebook cell 24.

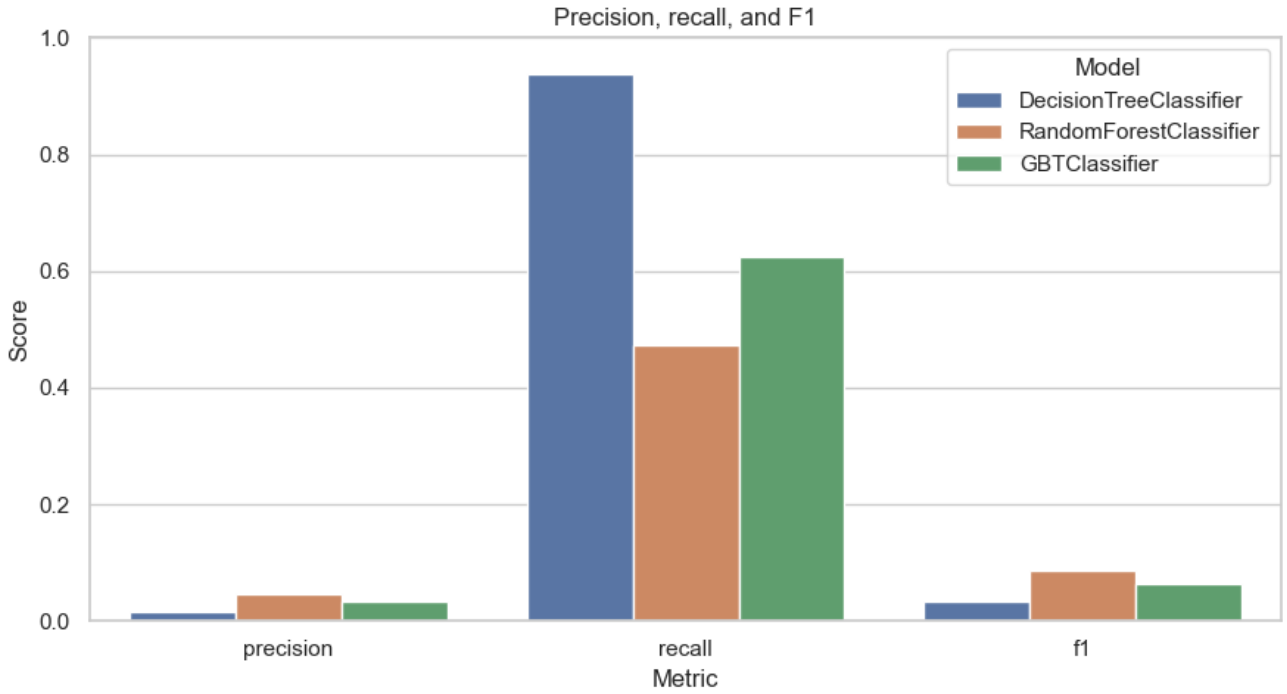


Figure 9. Inline chart output from notebook cell 24.

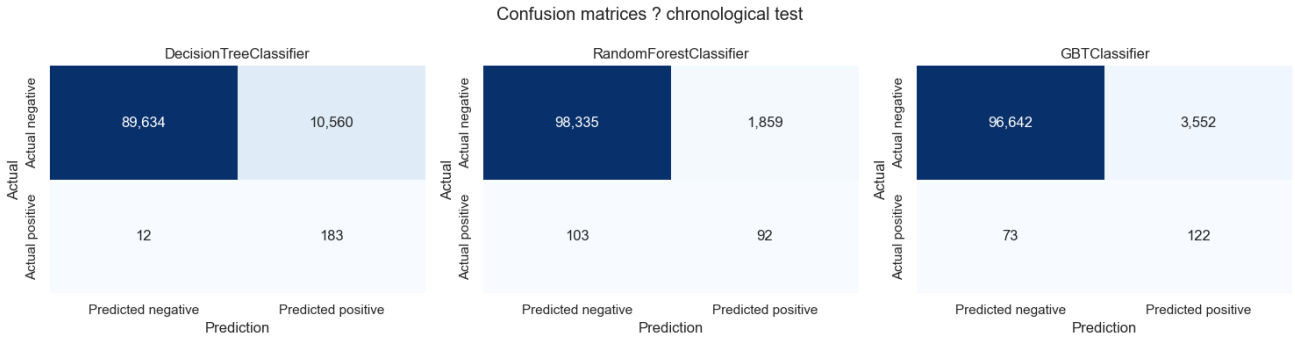


Figure 10. Inline chart output from notebook cell 24.

Notebook output tables

Prepared-data null summary

check	count
prepared_rows	500000
transaction_id_nulls	0
timestamp_nulls	0
sender_account_nulls	0
receiver_account_nulls	0
amount_nulls	0
payment_type_nulls	0
is_laundering_nulls	0
is_near_threshold_nulls	0

Amount-range feature summary

is_near_threshold	transaction_count	minimum_amount	average_amount	maximum_amount
0	495007	0.0000	3,587,389.00	84,853,140,000.00
1	4993	9,000.06	9,492.32	9,999.92

SQL output sizes

Notebook SQL output	Rows displayed
active_senders	20
aml_rate_by_payment_type	7
high_value_transactions	20
payment_types	20
sender_account_join	20
transaction_statistics	1
wildcard_payment_search	20

AML rate by payment type

payment_type	transaction_count	aml_positive	aml_rate
ACH	59038	439	0.0074
Cash	48278	14	0.0003
Bitcoin	14370	4	0.0003
Cheque	183571	32	0.0002
Credit Card	130415	21	0.0002
Wire	17004	0	0.0000
Reinvestment	47324	0	0.0000

High Value Transactions

transaction_id	sender_account	receiver_account	amount	payment_type
e0565449185c62fec25a8a55dbc168a59ced725410b45a...	004:806B9AF90	0116781:806B9B6A0	84,853,140,000.00	ACH
2762ab5f796daf6a9637530dd4b90dc775e44e300ce097...	015964:8026E7B20	0114590:805FBE280	69,431,620,000.00	Cheque
06ebe610f53707475e67659429d56a7d9e323ae80b1239...	014433:8023B9740	021:80368E630	58,678,780,000.00	Cash
117dea01ede0fcea4b9c46cfb4530e977d0a1b15847513...	0112497:804BB7A20	0014381:80551B160	45,882,280,000.00	Cheque
e47956ae7a3919c3c5ad8500b3da1af98baca506fca672...	029467:8039E0E70	009665:803A9D5B0	42,362,270,000.00	Cheque

First five rows of the saved notebook display; the full table remains in the notebook.

Active Senders

sender_account	transaction_count	total_sent
070:100428660	16544	3,865,869,000.00
070:1004286A8	9962	3,236,513,000.00
070:100428978	2033	757,945,800.00
070:1004286F0	1814	3,424,020,000.00
070:100428780	1714	15,893,680,000.00

First five rows of the saved notebook display; the full table remains in the notebook.

Payment Types

transaction_id	sender_account	receiver_account	amount	payment_type
e0565449185c62fec25a8a55dbc168a59ced725410b45a...	004:806B9AF90	0116781:806B9B6A0	84,853,140,000.00	ACH
2762ab5f796daf6a9637530dd4b90dc775e44e300ce097...	015964:8026E7B20	0114590:805FBE280	69,431,620,000.00	Cheque
06ebe610f53707475e67659429d56a7d9e323ae80b1239...	014433:8023B9740	021:80368E630	58,678,780,000.00	Cash

transaction_id	sender_account	receiver_account	amount	payment_type
117dea01ede0fcea4b9c46cfb4530e977d0a1b15847513...	0112497:804BB7A20	0014381:80551B160	45,882,280,000.00	Cheque
e47956ae7a3919c3c5ad8500b3da1af98baca506fca672...	029467:8039E0E70	009665:803A9D5B0	42,362,270,000.00	Cheque

First five rows of the saved notebook display; the full table remains in the notebook.

Transaction Statistics

transaction_count	total_amount	average_amount	minimum_amount	maximum_amount
500000	1,775,830,000,000.00	3,551,660.00	0.0000	84,853,140,000.00

First five rows of the saved notebook display; the full table remains in the notebook.

Sender Account Join

transaction_id	sender_account	amount	transaction_count	average_amount
58528390079edf2a0eefe445324b37ddb965ee8d159db4...	070:100428660	223,137,400.00	16544	233,671.99
65fa475bc7d12513c8b94f25dd7f32ff437bb201a2c27d...	070:100428660	180,488,900.00	16544	233,671.99
5213dda93f65639728dedddd90d2d18df6208fd8ae62b1...	070:100428660	160,331,800.00	16544	233,671.99
d7f29189284b5b0896e66313cd5d053a00bb31a1c02f22...	070:100428660	152,999,400.00	16544	233,671.99
cc54ccf431e4b027b4e0cf9e442455556c89b403cdb60f...	070:100428660	147,297,800.00	16544	233,671.99

First five rows of the saved notebook display; the full table remains in the notebook.

Wildcard Payment Search

transaction_id	sender_account	receiver_account	amount	payment_type
e0565449185c62fec25a8a55dbc168a59ced725410b45a...	004:806B9AF90	0116781:806B9B6A0	84,853,140,000.00	ACH
2762ab5f796daf6a9637530dd4b90dc775e44e300ce097...	015964:8026E7B20	0114590:805FBE280	69,431,620,000.00	Cheque
06ebe610f53707475e67659429d56a7d9e323ae80b1239...	014433:8023B9740	021:80368E630	58,678,780,000.00	Cash
117dea01ede0fcea4b9c46cfb4530e977d0a1b15847513...	0112497:804BB7A20	0014381:80551B160	45,882,280,000.00	Cheque
e47956ae7a3919c3c5ad8500b3da1af98baca506fca672...	029467:8039E0E70	009665:803A9D5B0	42,362,270,000.00	Cheque

First five rows of the saved notebook display; the full table remains in the notebook.

11. Results and Discussion

Values below are taken from the notebook's embedded metrics and split tables. Threshold-based measures use validation-selected cutoffs; PR-AUC and ROC-AUC use continuous test-set scores.

model	accuracy	precision	recall	f1	pr_auc	roc_auc	tp	tn	fp	fn	threshold
Decision Tree	0.8947	0.0170	0.9385	0.0335	0.0165	0.9129	183	89634	10560	12	0.9021
Random Forest	0.9805	0.0472	0.4718	0.0857	0.0315	0.9428	92	98335	1859	103	0.9073
Gradient-Boosted Trees	0.9639	0.0332	0.6256	0.0631	0.0291	0.9368	122	96642	3552	73	0.8754

Chronological split counts

split	rows	positive	negative	positive_rate
Train	299730	212	299518	0.0707%

split	rows	positive	negative	positive_rate
Validation	99881	103	99778	0.1031%
Test	100389	195	100194	0.1942%

Confusion counts

Model	TP	TN	FP	FN
DecisionTreeClassifier	183	89634	10560	12
RandomForestClassifier	92	98335	1859	103
GBTClassifier	122	96642	3552	73

In the saved notebook output, RandomForestClassifier has the highest F1 (0.0857); DecisionTreeClassifier has the highest recall (0.9385); RandomForestClassifier has the highest PR-AUC (0.0315); and RandomForestClassifier has the highest ROC-AUC (0.9428).

The Decision Tree's high recall is accompanied by many false positives. The Random Forest has the strongest F1 and ranking scores in this run but lower recall. Positive prevalence is low, so accuracy or ROC-AUC alone does not demonstrate useful real-world AML detection.

12. Conclusion and Future Work

The notebook demonstrates a local Spark workflow using synthetic AMLSim data, inline SQL results, three classifiers, and imbalance-aware evaluation. Results are limited by synthetic labels, selected features, one sampled dataset, and one chronological test period.

Future educational work could compare additional temporal periods, richer account/network features, calibration, and thresholds tied to a defined review capacity. Real-world use would require governed data, independent validation, compliance controls, and human oversight; none is implemented here.

References

[1] IBM AMLSim. github.com/IBM/AMLSim.

[2] IBM AMLSim Wiki. Download Example Data Set.

[3] Apache Spark 3.5.9 Documentation. spark.apache.org/docs/3.5.9.

[4] Project notebook: notebooks/01_aml_spark_workflow.ipynb.